Sentiment Classification of Reddit Resubmissions

CSE 158 Assignment 2

Zifan Yang
University of California, San Diego
La Jolla, CA
ziy086@ucsd.edu

Introduction

According to the statistics by Amazon's alexa.com, reddit.com is currently the sixth most visited website in the United States and the eighteenth most visited website globally as of November 11th, 2018. Due to the large number of users, it is inevitable that plenty of reddit posts contain content that has been posted before. This project will mainly focus on the public (users on the website) reception of redundant content and I will classify them into three categories based on the data analysis in the following section.

Dataset Analysis

In the reddit dataset from Himabindu Lakkaraju, Julian McAuley, and Jure Leskovec's study, there are 132,308 entries of image posts resubmission data with subreddit name, poster information, time of the post, title, scores, etc. The data set was taken from July 2008 to Jan 2013 and there were 16,736 unique images in all the submissions, averaging 7.9 resubmissions per image (Lakkaraju et al.). The dataset contains the following attribute:

- image_id: id of the image, submissions with the same id are of the same image
- unixtime: time of the submission (unix time)
- rawtime: raw text of the time
- title: submission title
- total_votes: number of upvotes + number of downvotes
- reddit_id: id of the submission on reddit, e.g. reddit.com/14c3ls
- number_of_upvotes: number of upvotes
- subreddit: subreddit, e.g. reddit.com/r/pics/
- number_of_downvotes: number of downvotes
- localtime: local time of the submission (unix time)
- score: number_of_upvotes – number_of_downvotes
- number_of_comments: number of comments the submission received
- username: name of the user who submitted the image e.g. www.reddit.com/user/thatseffedup

Reddit shows the upvote percentage near the total score in the comment page of each post. It gives the reader an idea about the quality of the post based on public opinions. Let's take this percentage and use it as a measurement of sentiment towards the post and I call it the public reception, which is essentially the ratio of upvotes to total votes:

$$R = \frac{number\ of\ upvotes}{total\ votes}$$

where total_votes is the sum of upvotes and downvotes. In other words, the higher the upvotes to total votes ratio, the more positive is the public reception. One obvious problem needs to be addressed is that subreddits might differ greatly in the number of subscribers. This can impact the reception because a subreddit with more subscribers will have a higher chance that someone will point out an image was reposted and lead to downvoting. To visualize this issue, let's first determine the popularity of the subreddits based on the number of submissions in that subreddit. Then, for each percentile (e.g. most popular of bottom 1 percent then 2, 3 etc.) of popularity, we compare their average R values to see if more popular subreddits give more negative reception to reposts (See Figure 1). Note that we want exclude posts with a total of zero downvotes and upvotes.

Figure 1:

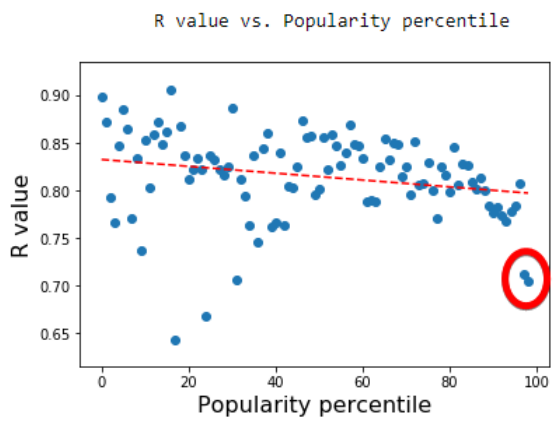


Figure 2:

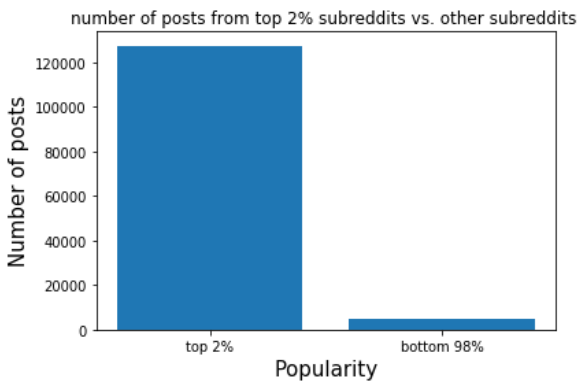


Figure 1 supported my hypothesis that higher popularity affects the R value negatively but not by much. The change in R was not significant until the top few percentiles. However, the points near the 100 percentiles (circled in red) is important because they make up of the majority of the posts as seen in Figure 2; therefore, when comparing the user reception from smaller subreddits to bigger ones, it is important to keep in mind that popularity significantly decreases the R value.

Also, as I was skimming through the dataset, I noticed that there are some reposts within the same day or week on the same subreddit, this seems to be affecting the R value significantly. To see if this effect applies to the dataset in general, let's plot the average R value based on number of days from the previous post. Since we don't know when the original post was (especially if it's before 2008), we take the resubmission with the minimal unix time as the first post, so it can be used as a reference for the later reposts.

Figure 3:

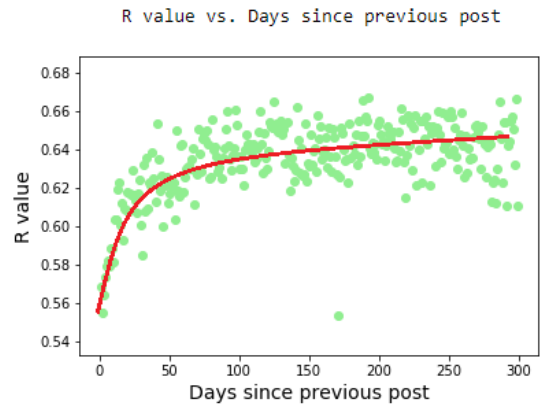
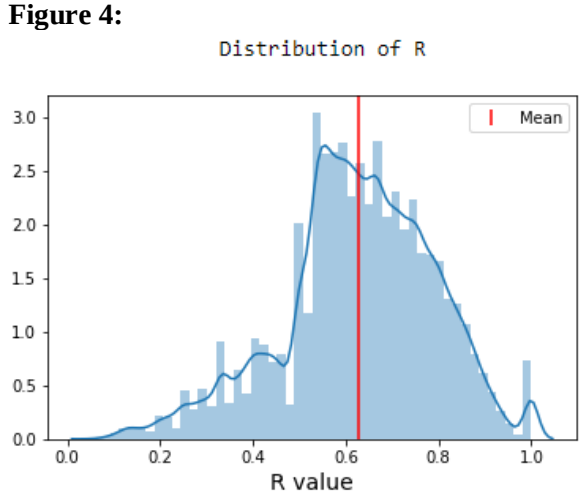


Figure 3 tells us that as time passes, people either forgot about a repost or people haven't seen the post upvoted them. The negative effect on user reception is especially significant within the first 50 days.

Next, let's see how titles affect the reception from users. Some content might be reposted but with a better title that grabs more attention. Some reposts might also have a more positive reception solely because of a funny title. First let's need to define what is considered "good" reception. The mean value of R across the entire dataset is 0.6274 (see Figure 4. Posts with zero total_votes are excluded).

Figure 4:



The standard deviation of R values is 0.1645, so let's consider posts within 1 standard deviation of the mean to be "average" and posts above and below 1 deviation from the mean to be "good" and "bad" respectively. After removing punctuation and converting all titles to lower case only, the top 20 most common words in all three categories good, bad and average seem to be about the same. All top 20 words are common stop words in English.

However, if we ignore the stop words we have the following result in Table 1.

Table 1:

bad	average	good
feel	feel	● xpost
reddit	like	feel
time	reddit	like
like	time	time
one	one	one
see	im	im
im	dont	reddit
get	day	see
dont	see	day
● facebook	get	get
day	gif	first
● found	every	dont
first	first	gif
think	ever	cat
fixed	think	every
post	cat	someone
new	oh	baby
today	xpost	think
gif	post	new
know	fuck	ever

With stop words ignored, we can see the results are more interesting. While lazy spelling like "im" and "dont" are still consist of stop words, most of the results are reasonable. The most obvious result is that the word "xpost" is the most popular word in the good titles. "xpost" refers to the same post cross posted to a different subreddit, therefore even though it's the same content, the audience are aware that the poster is trying to bring content from other communities into theirs, and this is usually well received by its community as seen from the data. On the contrary, reddit users seem to dislike content mentioning "facebook." For example, some titles containing the word facebook are "Just saw this on my Facebook feed" and "Found this on the Catholic Memes Facebook page," etc. These titles suggest that the content is taken from facebook. Notice the word "found" extremely close to "facebook" in terms of frequency, which also implies that the content was not original to the readers.

Lastly, some users repost more frequently than others. For example, the user "Gangsta_Raper" has 5608 posts alone. However, this is an extreme case as the second to fifth most active users only have 931, 858, 545 and 176 posts respectively. Furthermore, the above users have average R value across their posts within half of standard deviation of the average, therefore I won't be incorporating usernames in my feature design because of it and the fact there are 20,259 posts with username info missing.

This concludes the exploratory analysis of the dataset. The above information tells us certain features like title, timestamps and overall distribution of the posts across different subreddits can affect the public reception. In the next section, I'll use the gathered characteristics of the data to build feature vectors.

Predictive Task

As defined earlier, my measurement of public reception is the number of upvotes to total votes ratio R. High R value posts are more positively received by the public than low R value posts. Let's define the posts with R value that is one deviation above the mean to be "good," within one deviation above or below the mean to be "average" and more than one deviation below the mean to be "bad." Therefore, I will predict the public reception (sentiment) of a submission is either "good" ($R \geq 0.7919$), "bad" ($R \leq 0.4629$) or "average" ($0.4629 < R < 0.7919$) as mentioned in the first section.

According to the data analysis in the first section, words in title, time since previous post and popularity of a subreddit seem to be affecting public reception quite a lot. First let's preprocess the data. We need to remove the convert all titles to lowercase, remove all punctuations and remove stop words (like I did for Table 1). Let's also remove lazy spellings involving two stop words such as "im", "dont", "didnt", etc. We also want to divide the unix timestamp by 24*60*60 (number of seconds in a day). This converts the value to days which is easier for me to process when we build the feature vector. Lastly, compute R for each entry and label them as 0 for bad, 1 for average and 2 for good. After preprocessing the data, let's randomly divide the data into three equal portions to use as training set, validation set and test set. Since this is a classification problem of three classes, we'll evaluate the performance of our prediction based on classification accuracy:

$$Accuracy = \frac{number\ of\ correct\ predictions}{number\ of\ predictions}$$

For features, I will use the *tf-idf* weighted term frequencies from titles and some combinations of the following features:

$$feat1 = is\ the\ subreddit\ in\ top\ 2\%\ popularity$$
$$feat2 = days\ since\ the\ previous\ post$$
$$feat3 = does\ the\ tile\ contain\ "xpost"$$
$$feat4 = does\ the\ title\ contain\ "facebook"$$
$$feat5 = does\ the\ tile\ contain\ "found"$$

feat2 will be integer values and all other features are binary represented by 1 for true and 0 for false. Since this is a supervised multi-class classification problem, I will employ a combination of Naïve Bayes, logistic regression and Support Vector Machines and multiple variants of features mentioned above. In the next section, I will build two simple baselines using

$$v1 = [feat1, feat2]$$
$$v2 = [feat1, feat3, feat4, feat5]$$

and *tf-idf* term frequencies of the titles as my feature vectors. From there, I will pick the model with the best performance and continue to adjust the features and compare the results.

Model

In this section I will discuss the different models and features I used to approach the problem and how does their performance compare with each other. For Naïve Bayes, logistic regression and Support Vector Machines, I will use the following Python libraries from *sklearn*: *MultinomialNB*, *LogisticRegression* and *LinearSVC*, *tf-idf* term frequencies will be obtained using *TfidfVectorizer*.

Baseline

As mentioned in the previous section, let's employ the three different feature vectors on all three models. This should give me a general idea on which model is best for the task at hand and optimize that model incrementally. For *tf-idf*, I set the min_df (ignore terms that occurred in less than min_df documents) parameter to 5 due to memory issue caused by the large amount of text. I also set it to consider both unigrams and bigrams. Training the different models on our training set as described previously we get the following results on validation and test set:

LinearSVC(with C=1.0)

	validation set accuracy	test set accuracy
tf-idf	0.6633	0.6624
v1	0.6636	0.6625
v2	0.6779	0.6735

Multinomial Naïve Bayes

	validation set accuracy	test set accuracy
tf-idf	0.6887	0.6881
v1	0.6832	0.6829
v2	0.6823	0.6820

Multinomial Logistic Regression

	validation set accuracy	test set accuracy
tf-idf	0.6929	0.6922
v1	0.6836	0.6852
v2	0.6948	0.6943

Notice that *tf-idf* is doing worse than the other feature vectors in LinearSVC and about the same in the other two models. This tells me that term frequency is not a good indicator of public reception. The models also all had bad training error which indicates that they are underfitting. Overall, logistic regression seems to be the best model.

Combining the Logistic Regression Models

From the previous baseline we see that the number of days past since the previous repost certainly have some effect on the accuracy. In this section I'm going to combine *v2* with the *tf-idf* approach. Since word frequency is already accounted with *tf-idf*, I will not be using *v1* as part of this model. I will make use of the built-in *decision_function()* of the *sklearn* Python library to determine what to predict base on the confidence values of *v2* and *tf-idf* models. This method gives us an improved validation set accuracy of 0.6968 and test set accuracy of 0.6973. Still, only marginal improvements. Therefore, I want to further explore how words in title might affect R other than just their frequency.

Jaccard Similarity of Titles

A reader of the post would have a negative opinion about the post if they click on

the image and see that the content was old. A user might also downvote the post without looking at the content if they realize that the title is similar to some other posts they have seen before. We want to find the similarity of a post to all the previous posts of the same image. Let n be the number of posts before the post we are trying to compare, t_{new} be the title of the post and S be the similarity score.

$$S = \frac{1}{n}\sum_{i=1}^{n} Jaccard(t_{new}, t_i)$$

For this part I use the original titles with only the punctuation removed and converted to lower case without removing the stop words. Because stop words are important part of the English language, and since we are not simply counting term frequency, we want to include them. The average S across the entire date set is 0.1463 and the standard deviation of S is 0.1241. Let's check $S > 0.2704$ (mean + standard deviation) in to the feature vector *v2*, meaning if S is greater than 0.2704 then the title is highly unoriginal and potentially copied from past reposts. If the title is flagged as highly unoriginal, we put them into the "bad" category, otherwise we run the previous logistic regression model. This model gives a validation set accuracy of 0.7216 and test set accuracy of 0.7191. Overall a decent improvement since significantly less entries are considered "bad" or "good," therefore they are significantly harder to classify than "average."

Community Specific Similarities

An obvious issue with the previous model is that different subreddits have drastically different populations. As demonstrated in Figure 2 in the first section. Popularity affects the audience's reaction to the content, but this can also apply to unoriginal titles. Small or inactive communities' posts won't help in predicting sentiment in the larger communities where most of the resubmissions appear. Also, the way I categorized "reception" lead to a similar problem to the Amazon dataset in Assignment 1 where most of the entries is one category (women's clothing in Amazon dataset and "average" in our case). Therefore, for posts not in the top 2% subreddits as defined in section 1, we keep using the old model with S. For the top 2% subreddits, we use the following similarity S_c which is specific to each subreddit. Let n_c be the number of posts appeared in subreddit c before the current post with title $t_{new.}$

$$S_c = \frac{1}{n_c}\sum_{i=1}^{n_c} Jaccard(t_{new}, t_i)$$

Again, every title will have a value S_c. We append this to *v2* as $S_c >$ (mean + standard deviation of all the posts in that subreddit c). For example, for r/funny we have a value S_{funny} for each individual post $t_{new.}$ Let the number of posts in that subreddit be m. The average is $\frac{1}{m}\sum_{i=1}^{m} S_{c,tnew_i}$. Since there are 17 subreddits in the top 2%, we'll have a separate *v2* for each subreddit and predicting them separately. For the other subreddits that's not in the top 2%, we'll use the model in the previous section. The new model achieved a validation set accuracy of 0.7345 and test set accuracy of 0.7329.

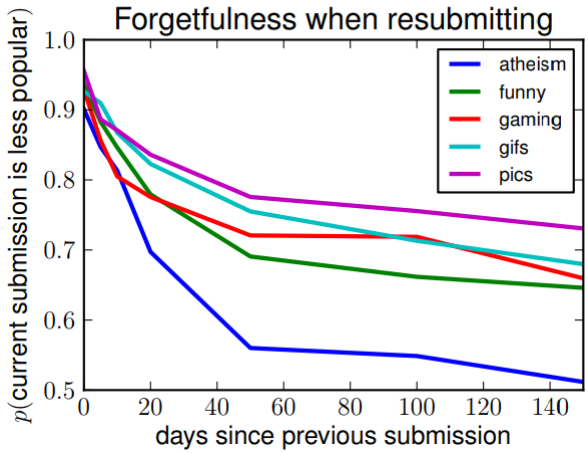
This is the best accuracy I was able to achieve so far employing what I've learned in class and some ideas from Professor McAuley's paper on the same dataset. In the next section I will talk about commonly used techniques in related works and the ideas I got from them.

Related Literature

The dataset I used is from "What's in a name? Understanding the interplay between titles, content, and communities in social media" by Professor McAuley, Lakkaraju and Leskovec. In their work, they predicted the normalized popularity of reddit resubmissions. They started with simple decay model based on the number of resubmissions and factors such as "forgetfulness," time of the submission, days since the previous submission (Lakkaraju et al.). Later, they incorporated more complex language features such as part of speech. While my predictive task is different from the popularity of a submission, we found similar correlation between a submission's success (either

public sentiment or the popularity) and time passed since the previous submission.

Figure 5 (Lakkaraju et al.):

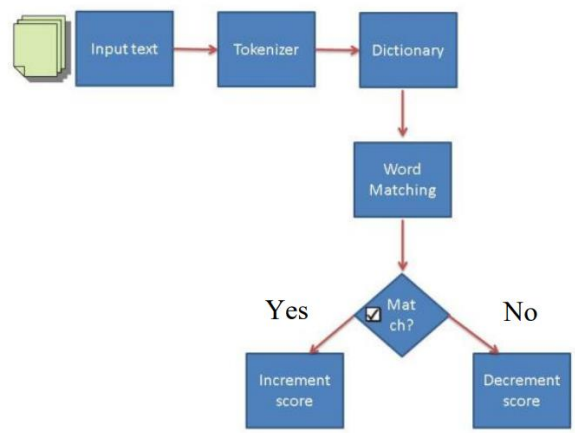


Comparing the graph of Figure 5 to Figure 3, we can see that the same trend holds where more days passed since the previous submission leads to better success of the post. In Professor McAuley's study, lower *p*in Figure 5 means better chance of a post to be more popular. Also, similarly, both Figure 3 and Figure 5 shows that around the 50 days mark, the decline of success slows down drastically.

The idea I was able to borrow from this study is the "originally of titles" measurement. In their work, they considered the Jaccard similarity of submissions overall and within each community. Their final model is a linear combination of all the factors. Similarly, I used my feature vector *v2* in the same way. While the goal of their study was to "understand when, where, and how a submission should be targeted," it is similar to my problem on determining how the public will react given a certain post (Lakkaraju et al.). The final result is also similar even though we measured "success" differently. We see that factors such as number of days since previous submission and similarity in title have effect on a submission's success, especially when considering community specific title similarities and part of speech.

Sentiment analysis of social media text has been a frequent topic of study. The method I used were mostly from the lectures of this course. In Harsh Thakkar and Dhiren Patel's paper titled "Approaches for Sentiment Analysis on Twitter: A State-of-Art study," they described a more advanced technique on studying the sentiment of Tweets. The main idea described by their work in "lexical analysis" (Thakkar and Patel). To summarize, lexical analysis uses a dictionary consists of pre-tagged lexicon and input text is tokenized and then matched for the dictionary (Thakkar and Patel). If a positive match is found, the score of that input text is increased (Thakkar and Patel).

Figure 6 (Thakkar and Patel):



This analysis is then combined with more traditional approaches such as Multinomial Naïve Bayes and Support Vector Machines to achieve higher accuracy than traditional supervised-learning-only approaches (Thakkar and Patel).

Conclusion

I've explored various features and models for the predictive task such as time, popularity, term frequencies in titles and similarity between titles. The results of the various models are summarized in the following table (Table 2):

Table 2:

	validation set accuracy	test set accuracy
LR with *tf-idf*	0.6929	0.6922
LR with *v1*	0.6836	0.6852
LR with *v2*	0.6948	0.6943
LR with *tf-idf* + *v2*	0.6968	0.6973
LR with *tf-idf* +[*v2*, *S* > *s*+*m*]	0.7216	0.7191

Last model + LR with *tf-idf* +[*v2*, $S_c > s+m$]	0.7345	0.7329

(where *s* is the standard deviation of S/S_c and *m* is the mean of S/S_c)

The baseline models all had similar performances with *tf-idf* and *v2* being slightly higher. Picking the most popular word in the "good" sentiment didn't work well. Because large amount of titles in the "average" sentiment also contains the word. They are not ranked as high in word count because the "average" sentiment contains significantly more posts. *tf-idf* solves this issue with *v1* since all term frequencies are considered and weighted properly. However, we see that the performance is comparable to *v2* which only uses popularity and time since previous submission as a feature vector. This shows that simple term frequency is not enough to determine the public sentiment. Combining *v2* and *tf-idf* showed marginal improvement but still shows that these features are not predictive enough.

The two models had the most successful results were the last two which considered the "originality" of titles. The first considers the originality across the whole training set while the second one considers the originality with in a specific popular community (namely the top 2% subreddits). These models worked well because users can not only notice resubmitted images but also unoriginal text. A user would downvote a title that's been used on the same content or a title that is generic and boring. I think the second one worked especially well because a user tends to frequent a few subreddits they like, thus resulting resubmissions being more easily spotted.

Overall, the public reception of a post seemed to be heavily determined by the audience's memory of a post. Features that increases the chance of the audience remembering a post have success on predicting their sentiment towards the post. Less time passed, similar title and popularity of the subreddit all increases the audience's chance to remembering content in question. In the future, I would try to incorporate part of speech tags as proposed in Professor's McAuley's paper. I should also consider modifying the time since previous submission to be community specific instead.

References

- www.alexa.com/siteinfo. Find Website Traffic, Statistics, and Analytics. 2018
- Harsh Thakkar and Dhiren Patel. Approaches for Sentiment Analysis on Twitter: A State-of-Art study. arXiv.org, 2015.
- Himabindu Lakkaraju, Julian McAuley, Jure Leskovec. What's in a name? Understanding the interplay between titles, content, and communities in social media. ICWSM, 2013.